Code.Hub

WARM-UP ASSIGNMENT

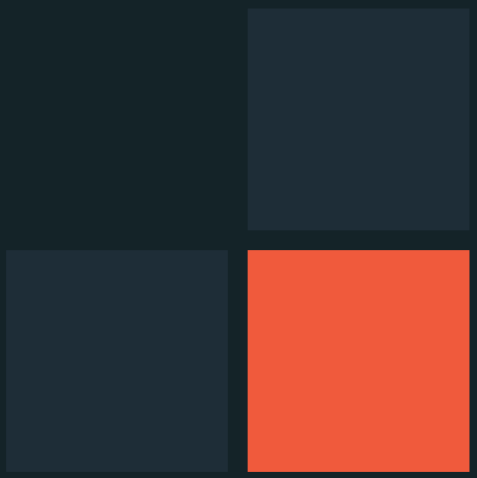
HelpDesk Lite

An Agentic IT Support Assistant for Team Onboarding

Team Warm-Up · Agentic AI Bootcamp

Code.Hub × PwC

November 2026



ABSTRACT

The challenge is to build a small assistant that receives an IT support request, gathers the missing facts, checks policies and known service outages, and proposes a single action that a human approves before it is executed.

This is a team warm-up, not a mini-capstone. Its purpose is to get your team working together, to set up a shared engineering stack, and to prove that each core component works end to end. Keep the business logic deliberately simple.

All company details below are fictional exercise assumptions.

8	12	20	2 weeks
TEAMS	APPLICATIONS	POLICY DOCS	TIMEBOX

01 The business case

Nimbus Analytics is a mid-sized consultancy with two offices and a five-person IT team. Support requests arrive by chat and email, and the IT team works across disconnected sources:

- IT policies and how-to guides.
- Employee directory and team membership.
- Service catalogue and application dependencies.
- Open incidents and outage notices.
- A simple ticketing system.

When an employee reports a problem, the support engineer must first ask for missing details, check whether a known outage explains it, look up the relevant policy, and only then decide what to do.

Many apparently individual problems are symptoms of a shared outage. A VPN complaint may be caused by a failing identity provider that affects dozens of colleagues, and resetting one password will not fix it.

> BUSINESS GOAL
>
> **The company wants to reduce the time to a correct first response while keeping a human in control of every action that changes an account or a system.**

Your team is asked to develop **HelpDesk Lite**, an assistant used by IT support engineers.

02 The request that drives the demo

The demonstration begins with this message:

> INCOMING MESSAGE · EMPLOYEE
>
> *"I can't connect to the VPN since yesterday afternoon and I need the finance reports before Friday. Can you reset my access?"*

The available data contains a few complications:

COMPLICATION	WHY IT MATTERS
The message does not say which device or office	The assistant must ask before acting, not guess.
An outage on the identity provider is open	The problem may not be individual; a reset would not help.
The VPN depends on the identity provider	The system must follow dependencies, not just match keywords.
Two versions of the password-reset policy exist	The document with the later effective date takes precedence.
The application restarts while approval is pending	The conversation must resume exactly where it stopped.

Keep the complications few and visible: the goal is to see each component working, not to stress-test it.

03 What the user should experience

An IT support engineer opens the chat interface and pastes the request. The application should:

01 **Receive the request** and extract structured fields: employee, application, symptom, and urgency.

02 **Ask for clarification** when an essential field is missing.

03 **Retrieve relevant policies** from the document store and cite the source document.

04 **Check the service graph** for dependencies and open outages that explain the symptom.

05 **Propose one action**, such as a credential reset, linking to an existing outage, or opening a ticket.

06 **Pause for human approval**, showing exactly what will be executed.

07 **Execute the approved action** in a simulated ticketing table and confirm the result.

The user must also be able to ask follow-up questions:

> *"Why didn't you reset the password?*

> *"Which other teams are affected by this outage?*

> *Which policy did you base this on?*

> *Continue the request I opened before the restart.*

04 Required architecture

Each technology must have a defined responsibility.

COMPONENT	RESPONSIBILITY
Docker Compose	One command starts every service. The same file becomes the base of your capstone repository.
Chainlit	Chat interface, progress messages, and approve or reject buttons.
LangGraph	The request workflow: extraction, routing, clarification, and approval interrupts.
LangChain Core	Prompts, model interface, tools, and structured output with Pydantic.
PostgreSQL	Ticket table and LangGraph checkpoints.
pgvector	Semantic search over the policy documents.
Neo4j	Employees, teams, applications, services, and outages.
LangSmith	Tracing of every run and a small evaluation dataset.

KEY PRINCIPLE

One graph, one model, simple tools. Multi-agent patterns, MCP integrations, additional vector stores, model training, and adversarial testing are out of scope for this exercise.

05 Team roles and ways of working

The warm-up is also the first time your team works together. Split the work so that everyone owns one part of the stack. Suggested roles:

ROLE	OWNS	FIRST MILESTONE
Platform	Docker Compose, environment template, startup instructions.	All services start with one command on every laptop.
Interface	Chainlit application and approval buttons.	A chat message reaches the graph and returns.
Workflow	LangGraph state, nodes, interrupts, and checkpointer.	A request pauses for approval and resumes after a restart.
Knowledge	Policy ingestion into pgvector and the Neo4j seed.	Both stores answer a test query.
Quality	LangSmith tracing and the evaluation dataset.	Ten test requests run and appear as traces.

Smaller teams combine roles. Every member must commit code and be able to explain the whole flow during the demo.

> TEAM AGREEMENT
>
> **Before writing code, agree on a branch strategy, a pull-request rule with at least one reviewer, a shared channel, and a definition of done. Record it in CONTRIBUTING.md.**

06 Starter data package

The trainer provides a small, ready-made dataset:

DATASET	SIZE
Employees with team and office	40
Teams	8
Applications and the services they depend on	12 applications · 10 services
Open outages	2
IT policies and how-to guides	20 documents
Evaluation requests with expected action	10

The data is clean except for two deliberate issues: an outdated policy version and one employee whose team is missing from the graph.

> PROVENANCE
>
> Every answer should cite its **source document** and **effective date**, and every action should be traceable in **LangSmith**. Build this habit from the first day.

07 Decision requirements

The assistant chooses exactly one of these actions:

A	B	C	D	E
Answer from policy only	**Link to an open outage**	**Reset credentials**	**Open a new ticket**	**Escalate to an engineer**

If an open **outage** explains the symptom, the assistant links the request to it instead of proposing a **credential reset**. The choice may be rule-based; no scoring or cost comparison is required.

APPROVAL RULE

Nothing that changes an account or creates a ticket runs without **explicit approval**. A rejected action must never execute, and a restart must not execute an approved action twice.

08 Deliverables and timeline

The team submits:

- A shared repository with docker-compose.yml, .env.example, and a README that starts everything in under ten minutes.
- The working Chainlit application covering the flow in Section 03.
- Policy ingestion and graph-seeding scripts.
- A LangSmith project with traces and one evaluation run over the ten requests.
- CONTRIBUTING.md with the team agreement.
- A ten-minute live demonstration, including a restart while approval is pending.

TIMELINE

MILESTONE	WHEN
Kick-off: roles, repository, team agreement	**Week 1 · first session**
Full stack running on every laptop	**End of week 1**
End-to-end flow with approval and resume	**Mid week 2**
Evaluation run and demo	**End of week 2**

WHY THIS MATTERS

This exercise is not graded. Its value is measured by how fast your team starts on the first capstone day: the repository you build here is the skeleton you will extend.